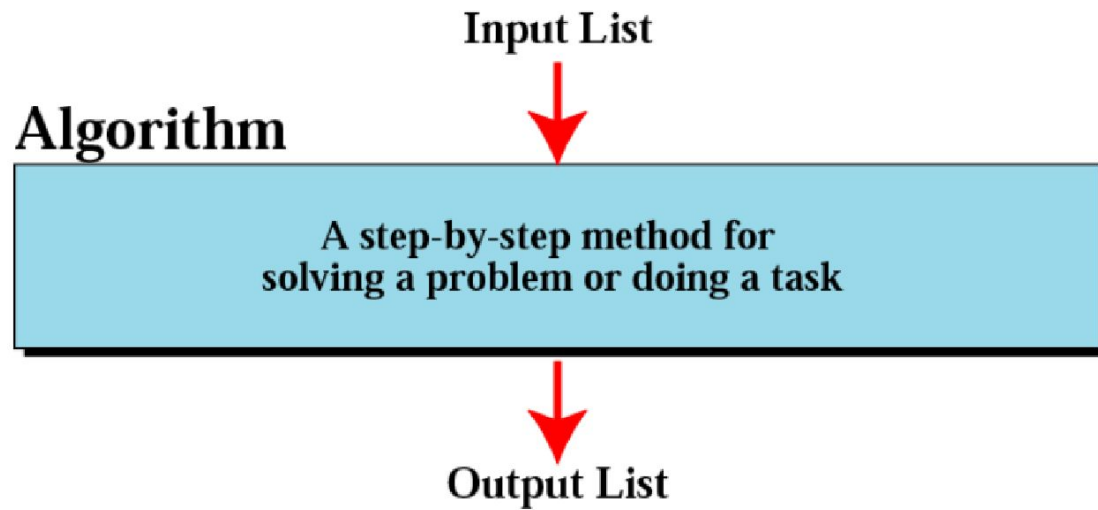


# Time Complexity

Maimuna Chowdhury

# Algorithm



# Algorithm Analysis:

**Algorithm analysis** is an important part of **computational complexity theory**, which provides theoretical estimation for the required resources of an algorithm to solve a specific computational problem. Analysis of algorithms is the determination of the amount of **time** and **space** resources required to execute it.

## Why the Analysis of Algorithm is important?

- **Performance:** How much time/memory/disk/etc. is used when a program is run. This depends on the machine, compiler, etc. as well as the code we write.
- **Complexity:** How do the resource requirements of a program or algorithm scale, i.e. what happens as the size of the problem being solved by the code gets larger.

# Why the Analysis of Algorithm is important?

**To predict the behavior of an algorithm** without implementing it on a specific computer.

- It is much more convenient to have simple measures for the **efficiency of an algorithm than to implement the algorithm and test the efficiency** every time a certain parameter in the underlying computer system changes.
- The analysis is thus only an approximation; it is not perfect.
- More importantly, by analyzing different algorithms, we can **compare them to determine the best one** for our purpose.

- let us say:
  - we run the Linear Search on a fast computer A and
  - Binary Search on a slow computer B and
  - pick the constant values for the two computers so that it tells us exactly how long it takes for the given machine to perform the search in seconds.
- Let's say the constant for A is 0.2 and the constant for B is 1000 which means that A is 5000 times more powerful than B.
- For small values of input array size  $n$ , the fast computer may take less time.
- **But, after a certain value of input array size, the Binary Search will definitely start taking less time compared to the Linear Search even though the Binary Search is being run on a slow machine.**

| Input Size | Running time on A | Running time on B |
|------------|-------------------|-------------------|
| 10         | 2 sec             | ~ 1 h             |
| 100        | 20 sec            | ~ 1.8 h           |
| $10^6$     | ~ 55.5 h          | ~ 5.5 h           |
| $10^9$     | ~ 6.3 years       | ~ 8.3 h           |

# Asymptotic Analysis

**Asymptotic Analysis:** is defined as the big idea that handles the above issues in analyzing algorithms. In Asymptotic Analysis, we **evaluate the performance of an algorithm in terms of input size** (we don't measure the actual running time). We calculate, how the time (or space) taken by an algorithm increases with the input size.

Asymptotic notation is a way to describe the running time or space complexity of an algorithm based on the input size.

# Asymptotic Analysis

The three most commonly used notations are Big O, Omega, and Theta.

1. **Big O notation (O):** This notation provides an **upper bound** on the growth rate of an algorithm's running time or space usage. It represents the **worst-case scenario**, i.e., **the maximum amount of time or space an algorithm may need to solve a problem**. For example, if an algorithm's running time is  **$O(n)$** , then it means that the running time of the algorithm increases linearly with the input size  $n$  or less.

# Asymptotic Analysis

2. **Omega notation:** This notation provides a **lower bound** on the growth rate of an algorithm's running time or space usage. It represents **the best-case scenario**, i.e., the **minimum amount** of time or space an algorithm may need to solve a problem. For example, if an algorithm's running time is  $\omega(n)$ , then it means that the running time of the algorithm increases linearly with the input size  $n$  or more.

# Asymptotic Analysis

3. Theta notation : This notation provides **both an upper and lower bound on the growth rate** of an algorithm's running time or space usage. It represents the **average-case scenario**, i.e., the amount of time or space an algorithm typically needs to solve a problem. For example, if an algorithm's running time is  $\theta(n)$ , then it means that the running time of the algorithm increases linearly with the input size  $n$ .

# Examples with their complexity analysis:

## **Constant Time Complexity $O(1)$ :**

In computer science,  $O(1)$  refers to constant time complexity, which means that the running time of an algorithm remains constant and does not depend on the size of the input. This means that the execution time of an  $O(1)$  algorithm will always take the same amount of time regardless of the input size. An example of an  $O(1)$  algorithm is accessing an element in an array using an index.

# Examples with their complexity analysis:

## Linear Time Complexity $O(n)$ :

The Time Complexity of a loop is considered as  $O(n)$  if the loop variables are incremented/decremented by a constant amount. For example following functions have  $O(n)$  time complexity. Linear time complexity, denoted as  $O(n)$ , is a measure of the growth of the running time of an algorithm proportional to the size of the input. In an  $O(n)$  algorithm, the running time increases linearly with the size of the input. For example, searching for an element in an unsorted array or iterating through an array and performing a constant amount of work for each element would be  $O(n)$  operations. In simple words, for an input of size  $n$ , the algorithm takes  $n$  steps to complete the operation.

```
for (int i = 1; i <= n; i = i + c) {  
    // some  $O(1)$  expressions  
}  
for (int i = n; i > 0; i = i - c) {  
    // some  $O(1)$  expressions  
}
```

# Examples with their complexity analysis:

## Quadratic Time Complexity $O(nc)$ :

The time complexity is defined as an algorithm whose performance is directly proportional to the squared size of the input data, as in nested loops it is equal to the number of times the innermost statement is executed. For example, the following sample loops have  $O(n^2)$  time complexity

```
for (int i = 1; i <= n; i += c) {  
  for (int j = 1; j <= n; j += c) {  
    // some  $O(1)$  expressions  
  }  
}
```

# Examples with their complexity analysis:

## Logarithmic Time Complexity $O(\log n)$ :

The time Complexity of a loop is considered as  $O(\log n)$  if the loop variables are divided/multiplied by a constant amount. And also for recursive calls in the recursive function, the Time Complexity is considered as  $O(\log n)$ .

```
for (int i = 1; i <= n; i *= c) {  
    // some  $O(1)$  expressions  
}  
for (int i = n; i > 0; i /= c) {  
    // some  $O(1)$  expressions  
}
```